TuHospedaje

INFORME DE SPRINT

Bitácora de Ejecución y Cierre — Sprint 3

TuHospedaje — Búsqueda, Disponibilidad, Favoritos y Contenido

Historias	US #22 a #29
Integrado	2026-06-04 (commit 7c29cf7)
Backend	139 tests en verde

Equipo de Desarrollo · Mayo 2026

BITÁCORA DE EJECUCIÓN Y CIERRE — SPRINT 3

Foco del Incremento: Búsqueda y Disponibilidad — Búsqueda por ciudad y fechas, Calendario de disponibilidad, Favoritos, Políticas, Galería de imágenes, Lucide Icons, Reseñas y Compartir en redes
Stack Tecnológico: Java 17 / Spring Boot 3.5 / Spring Security 6 / MariaDB / React 19 / Vite / Lucide React / SpringDoc OpenAPI

1. Resumen del Incremento (Scope)

El Sprint 3 se centró en dotar a la plataforma de funcionalidades de búsqueda avanzada, visualización de disponibilidad, gestión de favoritos, y contenido social. Se implementó un motor de búsqueda unificado con filtros dinámicos (ciudad, fechas, capacidad, precio, categoría) utilizando Specifications de Spring Data JPA, un sistema de reservas con control de concurrencia mediante optimistic locking, un calendario de disponibilidad visual, favoritos con toggle desde las cards del home, políticas de producto, reseñas con puntuación de estrellas, y la capacidad de compartir alojamientos en redes sociales. Adicionalmente, se incorporó documentación automática de la API mediante SpringDoc OpenAPI con Swagger UI.

Como mejoras complementarias, se implementó una galería de imágenes con modal viewer (navegación por teclado, contador, miniaturas), migración completa del sistema de íconos de Font Awesome a Lucide SVGs con un selector visual (IconPicker) en el panel de administración, rediseño del panel admin con sidebar + topbar + dashboard de estadísticas, y un componente CategoryCard con íconos Lucide en la página principal. El frontend recibió un polish visual general con design tokens globales (Inter, --radius, --shadow), header responsive con hamburger menu, y corrección de bugs de CSS scope.

2. Arquitectura del Sistema e Integración

2.1. Backend (Spring Boot + Spring Security 6)

Se expandió la arquitectura base con nuevos módulos y se refinaron los existentes:

```
Controller → Service (Interface + Impl) → Repository → Entity / DTO
```

Matriz de Componentes Introducidos:

Módulo	Entidad (Entity)	Objetos de Transferencia (DTO)	Capa de Servicio	Capa de Contro (Controller)
Reservas	`Reservation`	`CreateReservationRequest`, `ReservationResponse`, `AvailabilityResponse`, `OccupiedRange`	`ReservationService`	`ReservationContro`
Favoritos	*Relación en User*	—	*UserService extendido*	`FavoriteControlle`
Políticas	`Policy`	`PolicyDTO`	`PolicyService`	`PolicyController`
Reseñas	`Rating`	`RatingDTO`, `RatingRequest`	`RatingService`	`RatingController`
Subida imágenes	—	`UploadResult`	`CloudinaryService`	`UploadController`

- **Estrategia de Búsqueda:** Se reemplazó el enfoque de `@Query` JPQL por **Specifications** (Criteria API de Spring Data JPA). Esto permite componer filtros dinámicamente sin escribir strings SQL, manteniendo el tipado fuerte del compilador y la consistencia con el patrón del proyecto.
- **Control de Concurrencia:** Se implementó `@Version` (optimistic locking) en las entidades `Reservation` y `Lodging` para prevenir la doble reserva del mismo alojamiento en fechas solapadas. Si dos usuarios intentan reservar simultáneamente, el segundo recibe un HTTP 409 Conflict.
- **Búsqueda con Specifications:** La lógica de filtrado se construye en el servicio encadenando `Specification.and()`:
  - Filtro por ciudad (LIKE case-insensitive)
  - Filtro por capacidad de huéspedes
  - Filtro por categoría
  - Filtro por rango de precio
  - Exclusión de alojamientos con reservas solapadas (filtrado en Java post-query)
- **Autocompletado de Ciudades:** Endpoint `GET /api/lodgings/cities?q=` que devuelve ciudades distinct con coincidencia parcial. El frontend implementa debounce de 300ms.

2.2. Frontend (React + Vite)

Evolución de la SPA con nuevas páginas y componentes extraídos:

```
src/
├── components/
│   ├── Reservation/
│   │   └── ReservationModal.jsx    (Modal de confirmación de
reserva)
│   ├── ReviewsSection/
│   │   ├── ReviewsSection.jsx      (Sección de reseñas con
estrellas)
│   │   └── ReviewsSection.css
│   └── ShareModal/
│       ├── ShareModal.jsx          (Pop-up para compartir en
redes)
│       └── ShareModal.css
├── pages/
│   ├── Home/Home.jsx               (Buscador con autocompletado
+ fechas)
│   ├── SearchResults/
│   │   ├── SearchResults.jsx       (Página de resultados con
filtros)
│   │   └── SearchResults.css
│   ├── ProductDetail/
│   │   └── ProductDetail.jsx       (Calendario, precio,
reseñas, políticas)
│   ├── Favorites/
│   │   ├── FavoritesPage.jsx       (Lista de favoritos del
usuario)
│   │   └── FavoritesPage.css
│   └── Admin/
│       └── AdminPolicies.jsx       (CRUD de políticas en panel
admin)
├── hooks/
│   └── useAuth.js                  (Wrapper del contexto de
autenticación)
├── context/
│   └── AuthContext.jsx             (Lazy initialization del
estado de sesión)
```

```
└── services/
    └── api.js                          (Interceptor Bearer + manejo
de 401)
```

- **Extracción de Componentes:** La sección de reseñas se extrajo a `ReviewsSection`, reduciendo `ProductDetail.jsx` de 356 a 242 líneas.
- **Búsqueda con Transición (React 19):** Se utilizó `useTransition` para evitar warnings de `setState` síncrono en efectos, siguiendo las recomendaciones de React 19.
- **Calendario:** Se integró `react-datepicker` para la selección de fechas con fechas ocupadas deshabilitadas.
- **Fallback Visual:** Todas las imágenes tienen `onError` para mostrar placeholder si falla la carga, y `loading="lazy"` para rendimiento.
- **Galería con Modal Viewer:** Se reemplazó la grilla 50/50 por una imagen principal con miniaturas a la derecha. Al hacer click se abre un modal lightbox con navegación por flechas, teclado (`←`, `→`, `Esc`), contador (`N / M`) y cierre por overlay.
- **Migración a Lucide:** Se reemplazaron las clases CSS de Font Awesome por componentes SVG de Lucide React, tree-shakeables y consistentes visualmente. Se creó `iconMap.js` con mapeo de keys semánticas (`wifi`, `car`, `pool`, etc.) y un componente `IconPicker` para la selección visual desde el panel admin.
- **Favoritos Sincronizados:** Se agregó sincronización del estado de favoritos entre Home, SearchResults y ProductDetail mediante la prop `defaultFavorite` y un `useEffect` en `ProductCard`, eliminando la necesidad de recargar la página.
- **Admin Rediseñado:** Se reemplazó el sistema de tabs horizontales por un shell dedicado con sidebar vertical oscuro, topbar naranja y dashboard de estadísticas con cards clickeables y tabla de últimos alojamientos. El Header/Footer públicos se ocultan automáticamente en la ruta `/admin`.
- **CategoryCard:** Las categorías en el home dejaron de ser tags de texto plano y ahora son cards con ícono Lucide, nombre y descripción, con efecto hover y borde izquierdo decorativo.
- **Route Guard:** Se implementó un componente `RequireAdmin` que redirige a `/login` si el usuario no está autenticado y a `/` si no tiene rol ADMIN.

3. Trazabilidad de Historias de Usuario (User Stories)

ID	Historia de Usuario	Componente / Vista UI	Endpoint Backend	Criterio de Aceptación / Estado
US #22	Realizar búsqueda por ciudad y fechas.	`Home.jsx`, `SearchResults.jsx`	`GET /api/lodgings/search`, `GET /api/lodgings/cities`	Búsqueda unificada con filtros. Autocompletado de ciudades. Resultados con cards.
US #23	Visualizar disponibilidad en calendario.	`ProductDetail.jsx`	`GET /api/lodgings/{id}/availability`, `POST /api/reservations`	Calendario doble con fechas ocupadas deshabilitadas. Reserva con confirmación.
US #24	Marcar producto como favorito.	`ProductCard.jsx`	`POST /api/favorites/{lodgingId}`	Corazón visible solo para autenticados. Toggle con un clic.
US #25	Listar productos favoritos.	`FavoritesPage.jsx`	`GET /api/favorites`, `DELETE /api/favorites/{lodgingId}`	Lista de favoritos con opción de quitar.
US #26	Ver bloque de políticas del producto.	`ProductDetail.jsx`	Propiedad `lodging.policies`	Título subrayado. Políticas en columnas con ícono + título + descripción.
US #27	Compartir producto en redes sociales.	`ShareModal.jsx`	*N/A (Frontend)*	Pop-up con opciones Facebook, Twitter, WhatsApp. Imagen + descripción + enlace.

US #28	Puntuar producto con estrellas.	`ReviewsSection.jsx`	`POST /api/ratings`, `GET /api/ratings/lodging/{id}`	Sistema de 1-5 estrellas. Promedio dinámico. Reseñas con nombre, fecha, comentario.
US #29	Eliminar categoría con confirmación.	`AdminCategories.jsx`	`DELETE /api/categories/{id}`	ConfirmDialog reemplazó `window.confirm`

4. Catálogo de Endpoints Nuevos

4.1. Búsqueda y Disponibilidad

Método	Endpoint	Acceso (RBAC)	Descripción
GET	`/api/lodgings/search`	Público	Búsqueda unificada con filtros
GET	`/api/lodgings/cities`	Público	Autocompletado de ciudades
GET	`/api/lodgings/{id}/availability`	Público	Disponibilidad por rango de fechas
POST	`/api/reservations`	Autenticado	Crear reserva
GET	`/api/reservations/{id}`	Autenticado	Detalle de reserva

4.2. Favoritos

Método	Endpoint	Acceso (RBAC)	Descripción
POST	`/api/favorites/{lodgingId}`	Autenticado	Agregar favorito
DELETE	`/api/favorites/{lodgingId}`	Autenticado	Quitar favorito
GET	`/api/favorites`	Autenticado	Listar favoritos

4.3. Políticas

Método	Endpoint	Acceso (RBAC)	Descripción
POST	`/api/policies`	ADMIN	Crear política
PUT	`/api/policies/{id}`	ADMIN	Actualizar política
DELETE	`/api/policies/{id}`	ADMIN	Eliminar política
GET	`/api/policies`	Público	Listar políticas
GET	`/api/policies/{id}`	Público	Política por ID

4.4. Reseñas

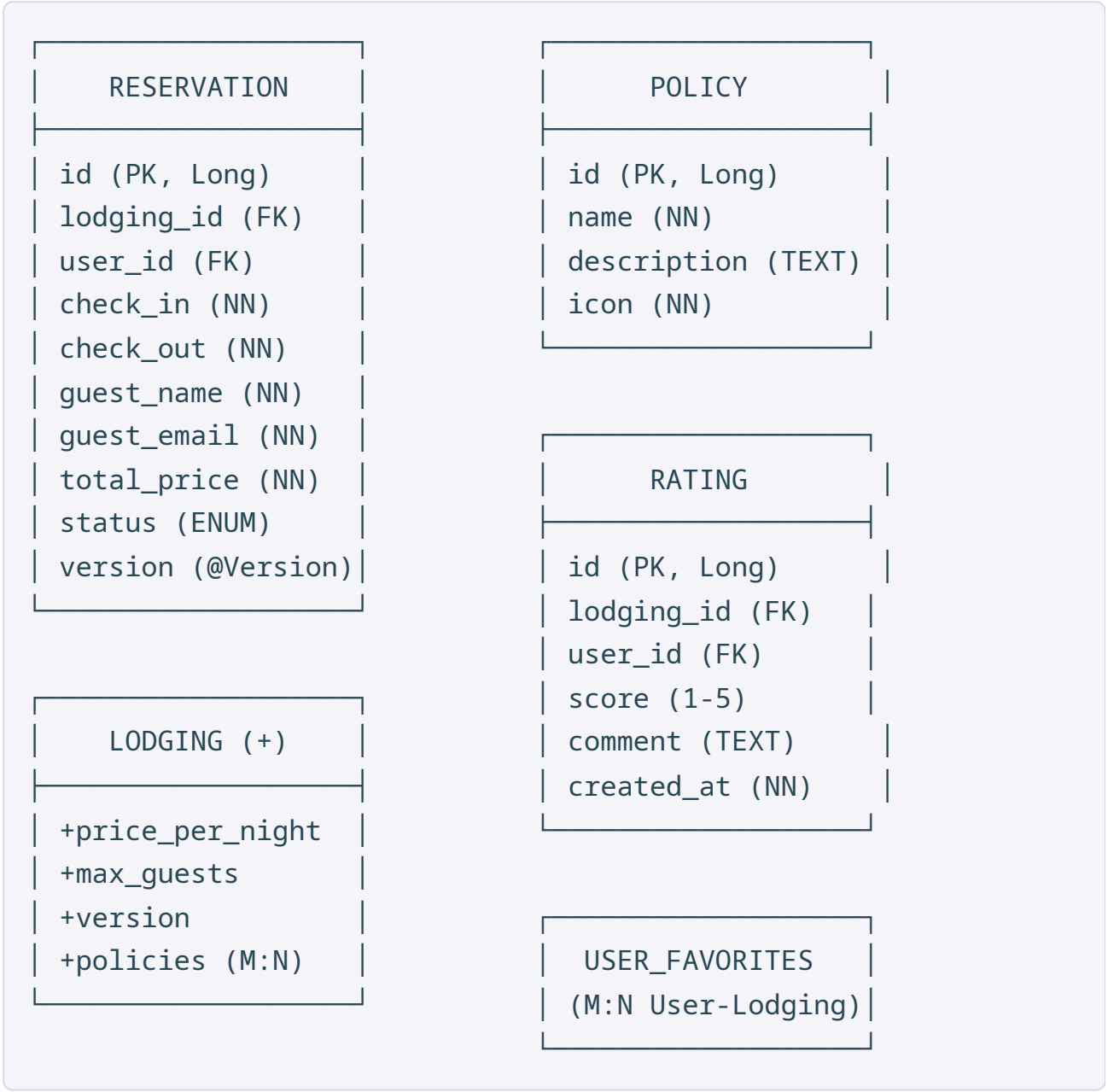
Método	Endpoint	Acceso (RBAC)	Descripción
POST	`/api/ratings`	Autenticado	Crear reseña
GET	`/api/ratings/lodging/{lodgingId}`	Público	Reseñas por alojamiento

4.5. Subida de Imágenes

Método	Endpoint	Acceso (RBAC)	Descripción
POST	`/api/upload`	ADMIN	Subir imagen a Cloudinary

5. Modelo de Datos

Nuevas Entidades



- **Reservation:** Nueva entidad central para el manejo de disponibilidad. Usa `@Version` para optimistic locking. FK a Lodging y User. El total se calcula como `días × pricePerNight`.
- **Policy:** Similar al patrón de Feature (catálogo reutilizable). Relación M:N con Lodging vía `lodging_policies`.
- **Rating:** Almacena reseñas con puntuación de 1 a 5. Relacionada con Lodging y User.

- **User:** Se agregó relación `@ManyToMany` con Lodging para favoritos vía `user_favorites`.
- **Lodging:** Se agregaron `pricePerNight`, `maxGuests`, y `@Version`. Nueva relación M:N con Policy. Nota: estos dos campos se agregaron como opcionales en Sprint 3; se volvieron obligatorios (con backfill) en un cambio posterior fuera de esta numeración de sprints, etiquetado "US #3" en el commit `d2b44f7` — no confundir con el US #3 de Sprint 1 (registro básico de alojamiento, ver `sprint-1-test-plan.md`).

6. Decisiones Técnicas Clave

- **Specifications vs @Query:** Se eligió Specifications (Criteria API) sobre `@Query` JPQL para mantener la consistencia con el patrón del proyecto (solo derived queries). Specifications permite componer filtros dinámicamente con tipado fuerte, aunque la exclusión de fechas ocupadas se realiza en Java post-query por no requerir `@Query`.
- **Optimistic Locking con @Version:** Se implementó `@Version` tanto en `Reservation` como en `Lodging`. La version solo en Reservation no previene inserts concurrentes, ya que las entidades nuevas arrancan con version=0. La version en Lodging provee un punto de contención único.
- **ConsoleEmailService:** Se modificó para loguear en consola en lugar de enviar emails reales, evitando consumir el límite gratuito de Mailtrap durante el desarrollo.
- **DTOs en Subcarpetas por Dominio:** Se reorganizó la estructura de `dto/` con subcarpetas por dominio (`lodging/`, `reservation/`, `auth/`, `category/`, `features/`, `user/`), mejorando la navegabilidad y escalabilidad.
- **Secretos en Variables de Entorno:** Todos los secrets (JWT, Cloudinary, BD) se movieron a variables de entorno con `.env` local en `.gitignore`, y `.env.example` con placeholders para el repositorio.
- **SpringDoc OpenAPI:** Se incorporó documentación automática de la API con Swagger UI en `/swagger-ui/index.html` y configuración personalizada con título y descripción del proyecto.
- **Lucide sobre Font Awesome:** Se eligió Lucide React por ser tree-shakeable (sin dependencia externa de CDN), liviano, y con soporte nativo de SVG en React. Font Awesome renderizaba strings de clases CSS como texto visible en la UI.
- **CSS Grid en Search Bar:** Se reemplazó `flex-wrap` por `display: grid` con `grid-template-columns: 1fr 1fr 1fr auto` para alinear los inputs de búsqueda, ya que los date inputs nativos tienen estilos que rompen la alineación en flex.
- **Admin Shell con Posicionamiento Fijo:** El admin panel usa `position: fixed; inset: 0` para evitar desbordamientos causados por el layout de la página principal y el margin por defecto del `body`.
- **CSS Scope en Vite:** Se corrigieron clases CSS genéricas (`.description`) que se filtraban a otras páginas, scopéandolas con el contenedor padre (`.product-detail`

`.description`).

7. Testing

- **139 tests backend:** Todos verdes (JUnit 5 + Mockito + MockMvc + Testcontainers con MariaDB 10.11). Incluyen unitarios e integración para CRUD de entidades, búsqueda con filtros, solapamiento de fechas, seguridad de endpoints y validación de DTOs.
- **Cobertura:** CRUD de todas las entidades, búsqueda con filtros, solapamiento de fechas, seguridad de endpoints, políticas, y validación de DTOs.
- **Frontend:** Sin test runner configurado. `npm run build` exitoso con 0 errores.

8. Limitaciones Conocidas y Deuda Técnica Controlada

1. **Refresh Tokens:** El sistema carece de refresh tokens. El JWT expira a las 8 horas forzando reautenticación. Pendiente para futura iteración.
2. **Precios por Temporada:** El precio por noche es fijo (`pricePerNight` en Lodging). No hay soporte para tarifas variables por temporada.
3. **Filtro por Features en Búsqueda:** El endpoint `findAvailable` acepta `featureIds` pero el frontend de SearchResults no lo expone como filtro.
4. **Notificaciones por Email:** Desactivadas en desarrollo (ConsoleEmailService en modo log). Requieren reactivar Mailtrap o configurar SMTP real para producción.
5. **Frontend sin Tests Automatizados:** No hay test runner configurado en el frontend. Las validaciones son manuales.
6. **Gestión de Reservas en Admin:** El panel admin no incluye una vista de reservas para moderación (cancelar, confirmar). Depende del backend de Sprint 4.
7. **Imágenes en Admin:** La subida de imágenes a Cloudinary desde el panel admin no está conectada a la UI de LodgingFormModal.

Anotación de estado actual

Este informe conserva el estado histórico del Sprint 3. La búsqueda vigente se muestra en `Home` y usa la consulta de la URL como fuente de verdad; la vista histórica de resultados ya no integra el flujo actual. Ante cambios o eliminación de la consulta, Home no debe mostrar resultados de una respuesta anterior.